

FERO-SENSE: Ferrous- Environment Robust Operation & Sensing Engine

FERO-SENSE: Ferrous-Environment Robust Operation & Sensing Engine

Safe and Efficient Operation of Mine Vehicles in Fog and Low-Visibility Conditions in Open Cast Iron Ore Mines (SIH26007)

Why Existing Solutions Fail (The Problem Gap)

System Architecture Overview

LAYER 1: HARDWARE — Detailed Design

1.1 Magnetic Anomaly Detection Array (MADA) — *THE DIFFERENTIATOR*

1.2 Seismic Proximity Array (SPA) — *SEES THROUGH GROUND*

1.3 Dust-Penetrating IR Camera System

1.4 Ultrasonic 3D Mesh Array

1.5 UWB Precision Positioning (GPS Replacement)

1.6 LoRa Mesh Network (V2V + V2I)

1.7 Driver Interface: HUD + Audio + Haptic

1.8 Complete Bill of Materials

LAYER 2: AI — Model Architectures

2.1 Magnetic Terrain Classification Network (MagNet-CNN)

2.2 Seismic Vehicle Identification Network (SeisID-Net)

2.3 Dust-Aware Image Enhancement (DAIE)

2.4 Transformer Multi-Sensor Fusion Engine

2.5 Bench Edge Detection — *SAFETY CRITICAL*

LAYER 3: SOFTWARE — Complete System

3.1 Software Architecture

3.2 Main Integration Loop

3.3 HUD Renderer

3.4 FOG-NET Protocol Implementation

3.5 Digital Twin — Terrain Map

3.6 Audio Engine

3.7 Haptic Controller

DEMO SETUP FOR SIH2026

Tabletop Mine Diorama

Demo Script (5 Minutes)

What Judges Will Ask — And Your Answers

Build Timeline (Post-Hackathon)

Summary: What Makes FERO-SENSE Genuinely Different

FERO-SENSE: Ferrous- Environment Robust Operation & Sensing Engine

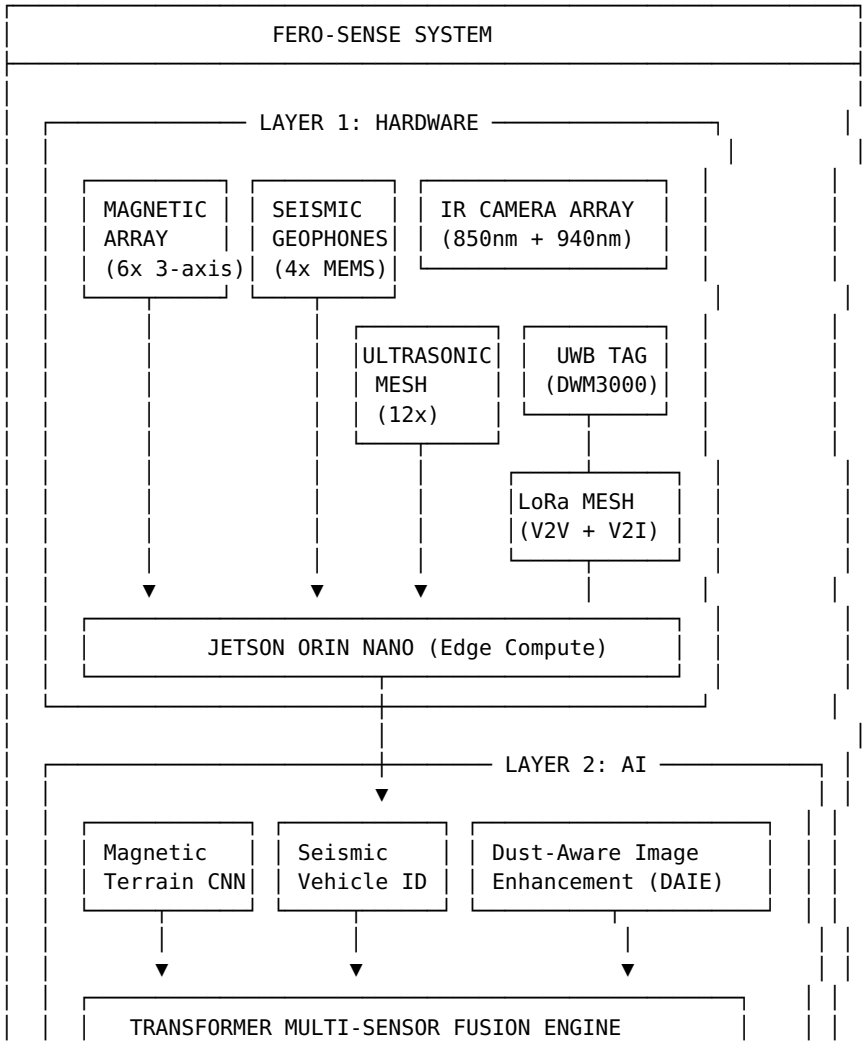
**Safe and Efficient Operation of Mine
Vehicles in Fog and Low-Visibility
Conditions in Open Cast Iron Ore Mines
(SIH26007)**

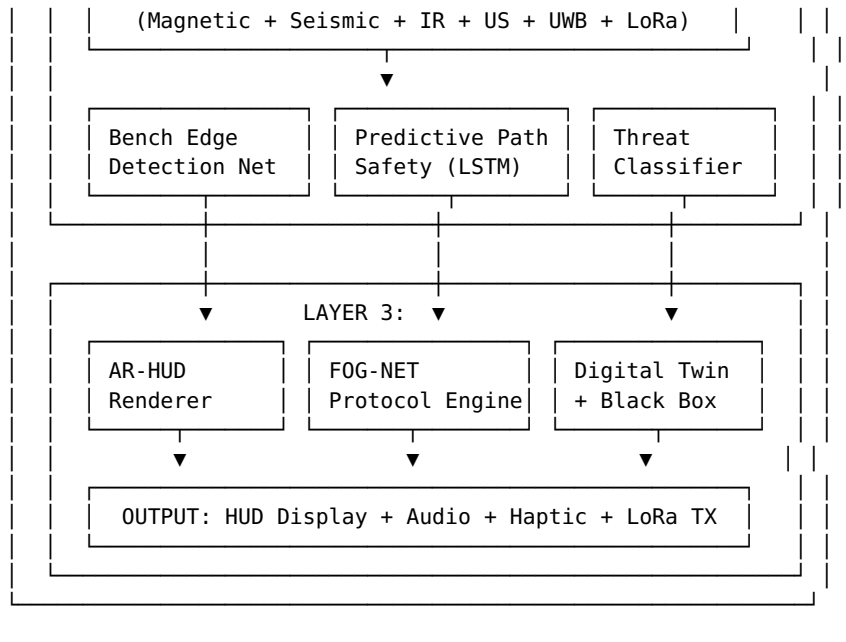
Why Existing Solutions Fail (The Problem Gap)

Existing Solution	Why It Fails in Iron Ore Mines
GPS/GNSS	Multipath from bench walls, signal loss in deep pits
LiDAR	Dust particles create massive point cloud noise
Standard Cameras	Zero visibility in dust-fog mix
Thermal Cameras	\$15K+ each, confused by hot equipment exhaust
Radar	False positives from ore piles, bench walls
Generic Dehazing	Not trained on iron-ore-dust optical properties
V2V Wi-Fi/4G	No coverage in open-pit depressions

Our Killer Insight: Iron ore mines are uniquely MAGNETIC environments. Ore dust, ore piles, and metal vehicles all create detectable magnetic signatures that are COMPLETELY UNAFFECTED by fog, dust, or darkness. Combined with ground-vibration sensing (seismic), we bypass the atmosphere entirely.

System Architecture Overview





LAYER 1: HARDWARE — Detailed Design

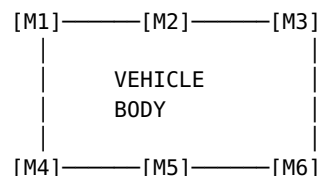
1.1 Magnetic Anomaly Detection Array (MADA) — *THE DIFFERENTIATOR*

Why This Works In Iron Ore Mines

- Iron ore (hematite/magnetite) has **1000x to 1,000,000x** the magnetic susceptibility of surrounding rock
- Road surfaces have LESS ore dust than piles → detectable magnetic gradient
- Bench edges show sudden magnetic drop-off (air has zero susceptibility)
- Metal vehicles create massive disturbances (dump truck = ~100 tons of steel)

Hardware Specification

MAGNETOMETER PLACEMENT (Top View of Vehicle):

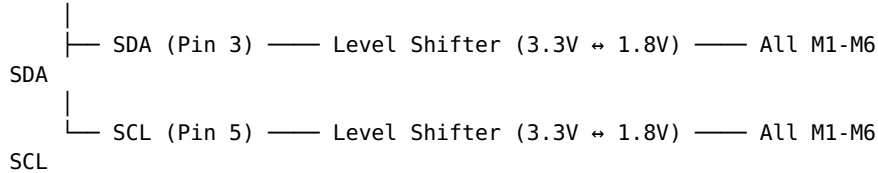


M1-M6: LSM303AGR 3-axis magnetometers

- Range: ± 49 Gauss (handles strong ore signatures)
- Resolution: 0.15 mGauss
- Sample rate: 100 Hz per axis
- Interface: I2C (shared bus, addresses 0x1E, 0x1D via address select)

Wiring Diagram

Jetson Orin Nano I2C Bus 1 (Pins 3, 5 on 40-pin header)



Power: 3.3V from Jetson → Each LSM303AGR VDD

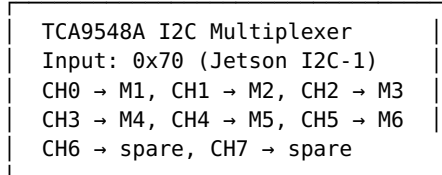
GND: Common ground bus

Address Multiplexing:

M1, M2, M3: SA0 pin HIGH → Address 0x1E

M4, M5, M6: SA0 pin LOW → Address 0x1D

(Use TCA9548A I2C multiplexer if more than 2 addresses needed)



What It Detects

Target	Magnetic Signature	Detection Range
Ore pile edge	Sudden increase in \	B\
Bench edge (drop)	Sudden decrease in all axes	1-3 meters
Road vs off-road	Baseline ~0.3G vs ~2G	Continuous
Approaching vehicle	Growing disturbance pattern	15-30 meters
Overhead power lines	50Hz oscillation (can filter)	N/A (noise)

1.2 Seismic Proximity Array (SPA) — SEES THROUGH GROUND

Principle

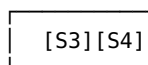
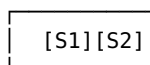
A 100-ton dump truck at 10 km/h creates ground vibrations propagating through rock/soil. These vibrations are: - **Independent of atmospheric visibility** - **Propagate 100-300m through ground** - **Have distinct frequency signatures per vehicle type**

Hardware

ACCELEROMETER PLACEMENT (Underside of vehicle chassis):

[Front Axle]

[Rear Axle]



S1-S4: ADXL354 (or ADXL355 for higher range)

- Range: ±2g / ±4g selectable

- Noise: 22.5 µg/√Hz

- Bandwidth: 1.5 kHz (plenty for vehicle vibrations)

- Interface: SPI (daisy-chainable)
- Mounted with hard mechanical coupling to chassis (no rubber isolation!)

SPI Connection to Jetson:

```

Jetson SPI1 (Pins 19, 21, 23, 24, 26)
|
├─ MOSI → S1.MOSI → S2.MOSI → S3.MOSI → S4.MOSI
├─ MISO ← S1.MISO (daisy chain: S1.MISO→S2.MOSI_in→...)
├─ SCK  → All SCLK
├─ CS0  → S1.CS
├─ CS1  → S2.CS (or use GPIO for CS)
└─ (Use 4 separate GPIO pins as chip selects)

```

Detection Capabilities

Vehicle Type	Freq Signature	Detection Range
Dump truck (100T)	8-25 Hz fundamental	200-300m
Excavator	15-40 Hz (hydraulic)	100-150m
Dozer	10-30 Hz (track)	150-200m
Light vehicle	20-60 Hz	50-80m

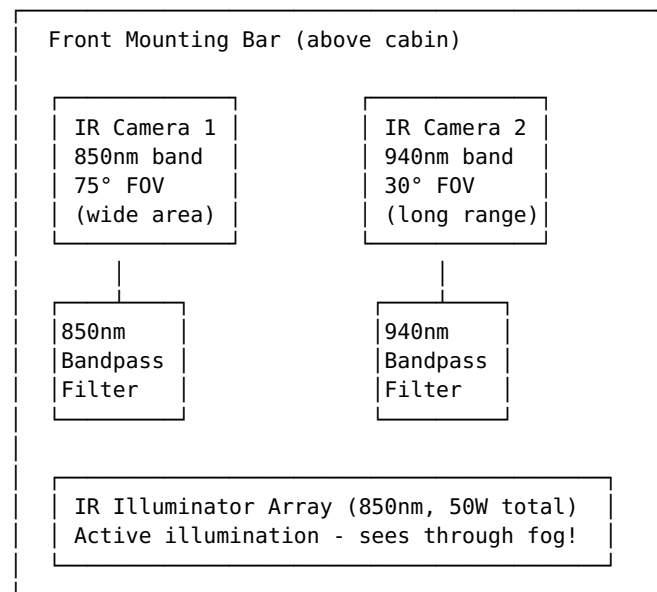
1.3 Dust-Penetrating IR Camera System

The Physics Trick

Iron ore dust (Fe₂O₃ particles, 1-100µm) has **lower absorption** at **850nm and 940nm** near-infrared compared to visible light. Scattering follows Mie theory — longer wavelength = less scattering for particles near wavelength size.

Hardware

DUAL IR CAMERA SETUP:



Cameras: Raspberry Pi HQ Camera + IR-cut filter removed
+ Hard bandpass filters (850nm ±10nm, 940nm ±10nm)
Interface: USB 3.0 to Jetson

Why Dual Wavelength

- ## 1.4 Ultrasonic 3D Mesh Array

1.5 UWB Precision Positioning (GPS Replacement)

[Bench Level 1] A1 — A2 — A3

 | | |

 Mine Road

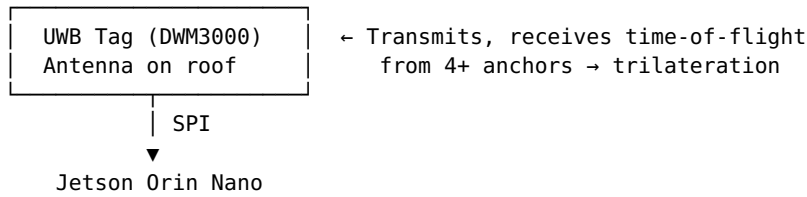
 | | |

[Bench Level 0] A4 — A5 — A6

 ↑

Loading Point

ON VEHICLE:



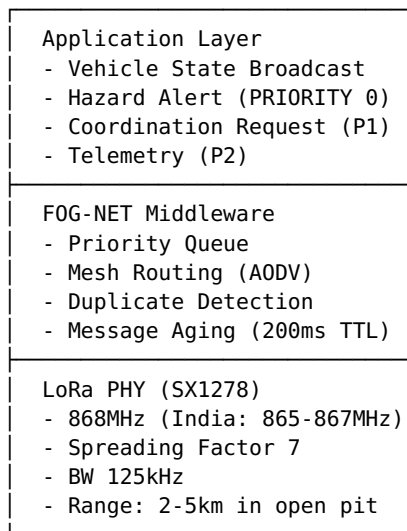
Position Calculation:

- Needs minimum 4 anchors in LOS
- TWR (Two-Way Ranging) protocol
- Output: (x, y, z) in mine coordinate frame
- Update rate: 10 Hz
- Range: up to 200m per anchor

1.6 LoRa Mesh Network (V2V + V2I)

Protocol Design: FOG-NET

FOG-NET PROTOCOL STACK:



Message Format (max 50 bytes for SF7):

Type	Prio	SrcID	Position	Velocity	Payload
1B	1B	2B	6B	4B	~36B

Vehicle Types in Payload:

- 0x01 = Dump Truck
- 0x02 = Excavator
- 0x03 = Dozer
- 0x04 = Light Vehicle
- 0x05 = UWB Anchor (infrastructure)

Hardware Connection

LoRa Module: SX1278 (Ra-02 board)

Interface: SPI (separate bus from accelerometers)

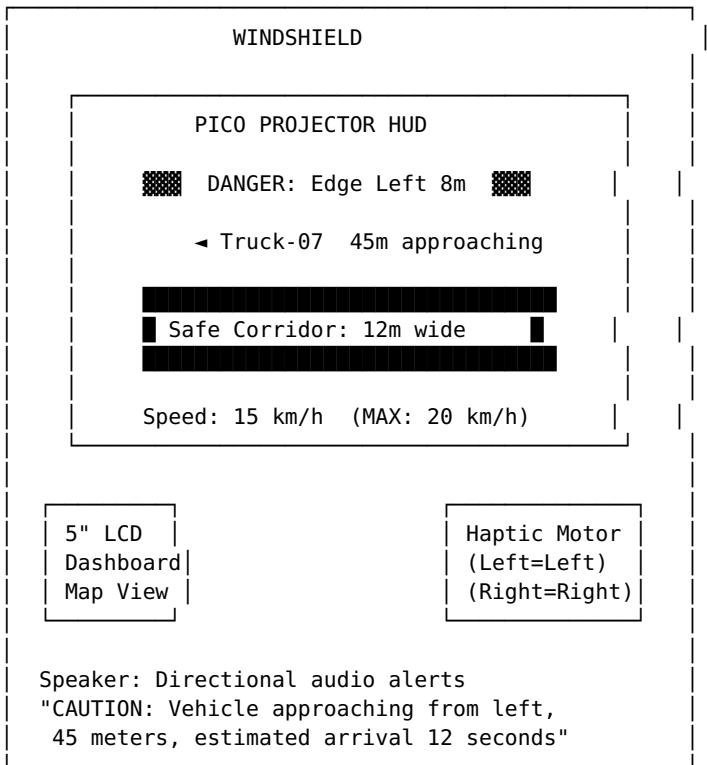
Jetson SPI2:

MOSI → RA-02.MOSI
MISO ← RA-02.MISO
SCK → RA-02.SCK
CS → GPIO 36 (manual CS)
DI00 → GPIO 38 (interrupt for RX/TX done)
RST → GPIO 40

Antenna: 868MHz whip antenna, mounted on vehicle roof
Power: 3.3V, ~120mA TX current

1.7 Driver Interface: HUD + Audio + Haptic

DRIVER CABIN INTERFACE:



- Components:
- Pico Projector: DLP2010 (\$50) or use 7" LCD as HUD display (\$40)
 - Display: 7" IPS LCD via HDMI (\$35)
 - Audio: MAX98357A I2S amplifier + 5W speaker (\$10)
 - Haptic: 2x ERM motor with DRV2605L haptic driver (\$15)
 - Left motor = left side threats
 - Right motor = right side threats
 - Pulse frequency increases with proximity

1.8 Complete Bill of Materials

Component	Qty	Unit Cost (₹)	Total (₹)	Source
Jetson Orin Nano 4GB	1	19,000	19,000	NVIDIA
LSM303AGR magnetometer	6	350	2,100	Mouser
TCA9548A I2C Mux	1	180	180	Amazon

ADXL354 accelerometer	4	800	3,200	Mouser
RPi HQ Camera (no IR cut)	2	2,500	5,000	Amazon
850nm bandpass filter	2	400	800	Thorlabs
940nm bandpass filter	2	400	800	Thorlabs
850nm IR LED array (50W)	1	1,500	1,500	Amazon
HC-SR04 ultrasonic	12	80	960	Amazon
DWM3000 UWB module	1	2,000	2,000	Digikey
DWM3000 UWB anchor	4	2,000	8,000	Digikey
SX1278 LoRa module (Ra-02)	2	250	500	Amazon
868MHz antenna	2	150	300	Amazon
7" IPS LCD + HDMI	1	2,500	2,500	Amazon
MAX98357A amp + speaker	1	350	350	Amazon
DRV2605L haptic driver	2	200	400	Amazon
ERM vibration motor	2	80	160	Amazon
Enclosure + wiring + PCB	1	3,000	3,000	-
Power supply (12V→5V, 12V→3.3V)	1	800	800	Amazon
TOTAL			₹51,550	

~\$620 USD — 1/50th the cost of commercial mining safety systems

LAYER 2: AI — Model Architectures

2.1 Magnetic Terrain Classification Network (MagNet-CNN)

Input

Shape: (6 sensors × 3 axes) = 18 channels
Window: 100 timesteps (1 second at 100Hz)
Input tensor: (18, 100)

Features per channel:
[X_mag1, Y_mag1, Z_mag1, X_mag2, Y_mag2, Z_mag2, ..., Z_mag6]

Architecture

```
import torch
import torch.nn as nn

class MagNetCNN(nn.Module):
    """
    Classifies terrain under/around vehicle using magnetic
    signatures.
    Classes: ROAD, ORE_PILE, BENCH_EDGE, EMPTY_SPACE,
    VEHICLE_PROXIMITY
    """
    def __init__(self):
        super().__init__()

        # Per-sensor feature extractors (6 sensors, shared weights)
```

```

self.sensor_encoder = nn.Sequential(
    nn.Conv1d(3, 32, kernel_size=7, padding=3), # 3 axes
    nn.BatchNorm1d(32),
    nn.ReLU(),
    nn.MaxPool1d(2),

    nn.Conv1d(32, 64, kernel_size=5, padding=2),
    nn.BatchNorm1d(64),
    nn.ReLU(),
    nn.MaxPool1d(2),

    nn.Conv1d(64, 64, kernel_size=3, padding=1),
    nn.BatchNorm1d(64),
    nn.ReLU(),
)
# Output per sensor: (64, 25) → Global Avg Pool → (64,)

# Spatial relationship between sensors
# Sensors have known positions on vehicle → use as
positional encoding
self.sensor_positions = torch.tensor([
    [-1.5, 2.0], # M1: front-left
    [0.0, 2.5], # M2: front-center
    [1.5, 2.0], # M3: front-right
    [-1.5, -2.0], # M4: rear-left
    [0.0, -2.5], # M5: rear-center
    [1.5, -2.0], # M6: rear-right
], dtype=torch.float32) # (x, y) in meters

self.position_encoder = nn.Linear(2, 64)

# Cross-sensor attention (which sensors agree/disagree?)
self.cross_attention = nn.MultiheadAttention(
    embed_dim=64, num_heads=4, batch_first=True
)

# Classification head
self.classifier = nn.Sequential(
    nn.Linear(64 * 6, 128),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 5), # 5 classes
)

# Edge distance regression (for BENCH_EDGE class)
self.edge_regressor = nn.Sequential(
    nn.Linear(64 * 6, 64),
    nn.ReLU(),
    nn.Linear(64, 1), # distance in meters
    nn.Softplus(), # always positive
)

def forward(self, x):
    # x: (batch, 18, 100) = (batch, 6_sensors * 3_axes,
    100_timesteps)
    batch = x.shape[0]

    # Split by sensor and encode each
    sensor_features = []
    for i in range(6):
        sensor_x = x[:, i*3:(i+1)*3, :] # (batch, 3, 100)
        feat = self.sensor_encoder(sensor_x) # (batch, 64, 25)
        feat = feat.mean(dim=-1) # Global avg pool → (batch,
        64)

        sensor_features.append(feat)

```

```

        # Add positional encoding
        pos_enc = self.position_encoder(self.sensor_positions) #
(6, 64)
        sensor_features = [f + pos_enc[i] for i, f in
enumerate(sensor_features)]

        # Stack: (batch, 6, 64)
        sensor_feat_stack = torch.stack(sensor_features, dim=1)

        # Cross-sensor attention
        attended, _ = self.cross_attention(
            sensor_feat_stack, sensor_feat_stack, sensor_feat_stack
        ) # (batch, 6, 64)

        # Flatten and classify
        flat = attended.reshape(batch, -1) # (batch, 384)
        terrain_class = self.classifier(flat)
        edge_distance = self.edge_regressor(flat)

        return terrain_class, edge_distance

```

Training Data Generation (For Prototype)

```

"""
Since we can't easily get real mine magnetic data for training,
we SYNTHESIZE it using physics-based models:

1. Dipole model for ore piles:  $B(r) = (\mu_0/4\pi) * [3(m \cdot \hat{r})\hat{r} - m] / |r|^3$ 
2. Background Earth field: ~0.5G with local anomalies
3. Vehicle signature: large dipole with slow movement
4. Bench edge: sharp transition from ore to air
5. Road: relatively flat, low anomaly

This gives us 10,000+ training samples in minutes.
"""

import numpy as np

def generate_magnetic_training_data(n_samples=10000):
    """Physics-based synthetic magnetic data"""
    samples = []
    labels = []
    edge_dists = []

    earth_field = np.array([0.25, 0.0, 0.43]) # India, ~45°
inclination

    for _ in range(n_samples):
        terrain = np.random.choice(['road', 'ore_pile',
'bench_edge', 'empty', 'vehicle'])

        # Base: earth field + sensor noise
        data = np.zeros((6, 3, 100)) # 6 sensors, 3 axes, 100
timesteps

        for s in range(6):
            # Add sensor noise (realistic: ±2mG)
            noise = np.random.normal(0, 0.002, (3, 100))

            if terrain == 'road':
                # Relatively flat, small random variations
                signal = np.random.normal(0, 0.005, (3, 100))

```

```

elif terrain == 'ore_pile':
    # Strong increase in |B|, direction changes
    dist_to_pile = np.random.uniform(1, 5)
    dipole_strength = np.random.uniform(0.5, 5.0) #
    # Simplified dipole decay
    magnitude = dipole_strength / (dist_to_pile ** 2)
    direction = np.random.randn(3)
    direction /= np.linalg.norm(direction)
    signal = np.outer(direction, np.ones(100)) *
    signal += np.random.normal(0, 0.01, (3, 100)) #

elif terrain == 'bench_edge':
    # Gradient across sensors: some see ore, some see
    edge_dist = np.random.uniform(0.5, 5.0)
    for ax in range(3):
        # Sensors closer to edge see transition
        base = earth_field[ax]
        gradient = np.linspace(0, -base * 0.3, 100)
        signal[ax] = base + gradient +
    np.random.normal(0, 0.003, 100)

elif terrain == 'empty':
    # Just earth field
    signal = np.outer(earth_field, np.ones(100))

elif terrain == 'vehicle':
    # Large, slowly varying disturbance
    freq = np.random.uniform(0.05, 0.2) # slow movement
    phase = np.random.uniform(0, 2*np.pi)
    t = np.linspace(0, 1, 100)
    magnitude = np.random.uniform(0.1, 0.5)
    signal = magnitude * np.sin(2 * np.pi * freq * t +
    np.random.uniform(0, 2*np.pi, (3,1)))

data[s] = signal + noise + np.outer(earth_field,
np.ones(100))

# Normalize relative to earth field
data = data - np.outer(earth_field, np.ones(100))

samples.append(data)

label_map = {'road': 0, 'ore_pile': 1, 'bench_edge': 2,
'empty': 3, 'vehicle': 4}
labels.append(label_map[terrain])

if terrain == 'bench_edge':
    edge_dists.append(np.random.uniform(0.5, 5.0))
else:
    edge_dists.append(0.0)

return np.array(samples), np.array(labels), np.array(edge_dists)

```

2.2 Seismic Vehicle Identification Network (SeisID-Net)

```

class SeisIDNet(nn.Module):
    """

```

```

        Identifies vehicle type AND estimates distance from ground
        vibrations.
        Input: 4 geophones × 3 axes × 200 timesteps (2 sec @ 100Hz)
        """
        def __init__(self):
            super().__init__()

            # 1D CNN for temporal feature extraction
            self.temporal_conv = nn.Sequential(
                nn.Conv1d(12, 64, kernel_size=15, padding=7), # 4*3=12
                nn.BatchNorm1d(64),
                nn.ReLU(),
                nn.MaxPool1d(4),

                nn.Conv1d(64, 128, kernel_size=11, padding=5),
                nn.BatchNorm1d(128),
                nn.ReLU(),
                nn.MaxPool1d(4),

                nn.Conv1d(128, 128, kernel_size=7, padding=3),
                nn.BatchNorm1d(128),
                nn.ReLU(),
            )

            # Frequency domain branch (FFT features)
            self.freq_conv = nn.Sequential(
                nn.Conv1d(12, 64, kernel_size=7, padding=3),
                nn.BatchNorm1d(64),
                nn.ReLU(),
                nn.AdaptiveAvgPool1d(32),
            )

            # Fusion
            self.fusion = nn.Sequential(
                nn.Linear(128 * 12 + 64 * 32, 256), # temporal + freq
                nn.ReLU(),
                nn.Dropout(0.3),
                nn.Linear(256, 128),
                nn.ReLU(),
            )

            # Task heads
            self.vehicle_classifier = nn.Linear(128, 5) # dump,
            excavator, dozer, light, none
            self.distance_regressor = nn.Sequential(
                nn.Linear(128, 64),
                nn.ReLU(),
                nn.Linear(64, 1),
                nn.Softplus(),
            )
            self.direction_regressor = nn.Sequential(
                nn.Linear(128, 32),
                nn.ReLU(),
                nn.Linear(32, 2), # (sin(θ), cos(θ)) for angle
            )

        def forward(self, x):
            # x: (batch, 12, 200)

            # Time domain
            temporal = self.temporal_conv(x) # (batch, 128, 12)
            temporal = temporal.reshape(temporal.shape[0], -1)

            # Freq domain (approximate with 1D conv on raw signal)
            freq = self.freq_conv(x)

```

```

freq = freq.reshape(freq.shape[0], -1)

# Fuse
fused = torch.cat([temporal, freq], dim=-1)
features = self.fusion(fused)

vehicle_type = self.vehicle_classifier(features)
distance = self.distance_regressor(features)
direction = self.direction_regressor(features)

return vehicle_type, distance, direction

```

2.3 Dust-Aware Image Enhancement (DAIE)

```

class DustAwareEnhancement(nn.Module):
    """
    NOT generic dehazing. Specifically models iron ore dust
    scattering.

    Physics: Iron ore dust (Fe2O3, 1-50um particles)
    - Scattering efficiency  $Q_{sca}$  peaks when particle size  $\approx$ 
    wavelength
    - At 850nm: moderate scattering (particles 0.4-4um scatter most)
    - At 940nm: less scattering (fewer particles in resonance)
    - KEY: The DIFFERENCE between 850nm and 940nm reveals dust
    density!

    Transmission model:
     $I_{obs} = J * T + A * (1 - T)$ 
    where  $T = \exp(-\beta * d)$ ,  $\beta$  depends on wavelength and dust density

    Since we have TWO wavelengths:
     $\beta_{850} > \beta_{940}$  (always, for iron ore dust)
    → We can solve for  $\beta$  and  $d$  separately → TRUE dehazing
    """
    def __init__(self):
        super().__init__()

        # Dust density estimator from dual-wavelength difference
        self.dust_estimator = nn.Sequential(
            nn.Conv2d(6, 32, 3, padding=1), # 2 wavelengths × 3
            (RGB→grayscale pairs)
            nn.ReLU(),
            nn.Conv2d(32, 32, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 3, 1), #  $\beta_r, \beta_g, \beta_b$  per pixel
            (transmission map params)
            nn.Softplus(), #  $\beta$  must be positive
        )

        # Atmospheric light estimator (ore dust has reddish-brown A)
        self.A_estimator = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(3, 16),
            nn.ReLU(),
            nn.Linear(16, 3),
            nn.Sigmoid(), # A in [0, 1] range
        )

        # Residual refinement (what the physical model misses)
        self.refinement = nn.Sequential(

```

```

        nn.Conv2d(6, 64, 3, padding=1),
        nn.ReLU(),
        nn.Conv2d(64, 64, 3, padding=1),
        nn.ReLU(),
        nn.Conv2d(64, 3, 3, padding=1),
        nn.Tanh(), # residual in [-1, 1]
    )

    def forward(self, img_850, img_940):
        """
        img_850: (B, 3, H, W) - 850nm image (grayscale replicated to
3ch)
        img_940: (B, 3, H, W) - 940nm image
        """
        # Concatenate both wavelengths
        x = torch.cat([img_850, img_940], dim=1) # (B, 6, H, W)

        # Estimate dust transmission parameters
        beta = self.dust_estimator(x) # (B, 3, H, W)

        # Estimate atmospheric light (dust color)
        A = self.A_estimator(x) # (B, 3)
        A = A.unsqueeze(-1).unsqueeze(-1) # (B, 3, 1, 1)

        # Physical dehazing: J = (I - A) / T + A
        # T = exp(-beta * d), we use beta as beta*d product
        T = torch.exp(-beta) # (B, 3, H, W)
        T = torch.clamp(T, 0.1, 1.0) # prevent division by zero

        # Apply to 850nm image (better signal)
        J_physical = (img_850 - A) / T + A
        J_physical = torch.clamp(J_physical, 0, 1)

        # Residual refinement
        residual = self.refinement(x) * 0.1 # small correction
        J_enhanced = J_physical + residual
        J_enhanced = torch.clamp(J_enhanced, 0, 1)

        return J_enhanced, T, A # enhanced image + transmission map
+ dust info

```

Training Strategy for DAIE

Training data generation (NO real foggy mine images needed):

1. Get CLEAR infrared images of mine environment (can be simulated)
2. Apply PHYSICS-BASED degradation:
 - Generate random dust density field (Perlin noise for realism)
 - Apply wavelength-dependent scattering: $\beta(\lambda) = C * \lambda^{(-\alpha)}$ *
dust_density
 - For Fe203: $\alpha \approx 2.5$ (Mie regime)
 - Add realistic sensor noise
3. Train network to recover clear image
4. Loss = L1(recovered, clear) + perceptual_loss +
physical_consistency_loss

The physical_consistency_loss ensures $T_{850} < T_{940}$ always (physics constraint)

2.4 Transformer Multi-Sensor Fusion Engine

```
import torch
```

```

import torch.nn as nn
import math

class SensorToken(nn.Module):
    """Converts each sensor modality into a token for transformer
    fusion"""
    def __init__(self, input_dim, embed_dim):
        super().__init__()
        self.proj = nn.Linear(input_dim, embed_dim)
        self.modalityEmbed = nn.Parameter(torch.randn(1, 1,
embed_dim) * 0.02)

    def forward(self, x):
        # x: (batch, input_dim)
        token = self.proj(x).unsqueeze(1) # (batch, 1, embed_dim)
        token = token + self.modalityEmbed
        return token

class MultiSensorFusionTransformer(nn.Module):
    """
    Fuses ALL sensor modalities into unified 360° threat map.

    Input tokens (one per modality):
    1. Magnetic terrain: (terrain_class_probs=5, edge_dist=1) = 6
    2. Seismic vehicle: (vehicle_type_probs=5, distance=1,
direction=2) = 8
    3. IR detected objects: (n_objects * (class=5, dist=1, dir=2,
conf=1)) → max 32
    4. Ultrasonic distances: 12 values + min distance = 13
    5. UWB position: (x=1, y=1, z=1, quality=1) = 4
    6. LoRa vehicles: (n_vehicles * (id=1, dist=1, dir=2, type=1)) →
max 20
    7. Vehicle state: (speed=1, heading=1, gear=1) = 3

    Output: 360° threat map (72 bins × 4 channels: threat_level,
type, distance, confidence)
    """
    def __init__(self, embed_dim=128, n_heads=8, n_layers=4,
n_output_bins=72):
        super().__init__()
        self.n_output_bins = n_output_bins
        self.embed_dim = embed_dim

        # Tokenizers for each modality
        self.mag_token = SensorToken(6, embed_dim)
        self.seis_token = SensorToken(8, embed_dim)
        self.ir_token = SensorToken(32, embed_dim) # max object
features

        self.us_token = SensorToken(13, embed_dim)
        self.uwb_token = SensorToken(4, embed_dim)
        self.lora_token = SensorToken(20, embed_dim)
        self.ego_token = SensorToken(3, embed_dim)

        # Learnable "empty slot" tokens (for variable number of
IR/LoRa objects)
        self.empty_token = nn.Parameter(torch.randn(1, 1, embed_dim)
* 0.02)

        # Positional encoding for angular bins
        angle_pos = torch.arange(n_output_bins).float() /
n_output_bins * 2 * math.pi
        self.angle_encoding = nn.Parameter(torch.randn(1,
n_output_bins, embed_dim) * 0.02)

        # Transformer encoder
        encoder_layer = nn.TransformerEncoderLayer(

```

```

        d_model=embed_dim,
        nhead=n_heads,
        dim_feedforward=embed_dim * 4,
        dropout=0.1,
        activation='gelu',
        batch_first=True,
    )
    self.transformer = nn.TransformerEncoder(encoder_layer,
num_layers=num_layers)

    # Output heads
    self.threat_head = nn.Sequential(
        nn.Linear(embed_dim, 64),
        nn.ReLU(),
        nn.Linear(64, 1),
        nn.Sigmoid(), # threat level [0, 1]
    )

    self.type_head = nn.Linear(embed_dim, 5) # obstacle type
    self.dist_head = nn.Sequential(
        nn.Linear(embed_dim, 64),
        nn.ReLU(),
        nn.Linear(64, 1),
        nn.Softplus(), # distance in meters
    )
    self.conf_head = nn.Sequential(
        nn.Linear(embed_dim, 64),
        nn.ReLU(),
        nn.Linear(64, 1),
        nn.Sigmoid(), # confidence [0, 1]
    )

def forward(self, mag_feat, seis_feat, ir_feat, us_feat,
            uwb_feat, lora_feat, ego_feat):
    batch = mag_feat.shape[0]

    # Create tokens
    tokens = [
        self.mag_token(mag_feat),
        self.seis_token(seis_feat),
        self.ir_token(ir_feat),
        self.us_token(us_feat),
        self.uwb_token(uwb_feat),
        self.lora_token(lora_feat),
        self.ego_token(ego_feat),
    ]

    # Add angle query tokens (what we want to know)
    angle_queries = self.angle_encoding.expand(batch, -1, -1)

    # Combine: sensor tokens as context, angle queries as target
    sensor_tokens = torch.cat(tokens, dim=1) # (batch, 7,
embed_dim)
    all_tokens = torch.cat([angle_queries, sensor_tokens],
dim=1) # (batch, 79, embed_dim)

    # Transform
    transformed = self.transformer(all_tokens) # (batch, 79,
embed_dim)

    # Extract angle query outputs
    angle_output = transformed[:, :self.n_output_bins, :] #
(batch, 72, embed_dim)

    # Generate threat map
    threat = self.threat_head(angle_output).squeeze(-1) #

```

```

(batch, 72)
    obj_type = self.type_head(angle_output) # (batch, 72, 5)
    distance = self.dist_head(angle_output).squeeze(-1) #
(batch, 72)
    confidence = self.conf_head(angle_output).squeeze(-1) #
(batch, 72)

    threat_map = {
        'threat': threat,
        'type': obj_type,
        'distance': distance,
        'confidence': confidence,
    }

    return threat_map

```

2.5 Bench Edge Detection — **SAFETY CRITICAL**

```

class BenchEdgeDetector(nn.Module):
    """
    Fuses multiple signals for bench edge detection:
    - Magnetic gradient (sudden drop at edge)
    - UWB position vs known terrain map
    - Ultrasonic front sensors (if edge is close)

    This is the MOST CRITICAL safety feature - falling off benches
    is #1 killer.
    """
    def __init__(self):
        super().__init__()

        # Magnetic gradient analyzer
        self.mag_gradient = nn.Sequential(
            nn.Linear(18, 32), # 6 sensors × 3 axes differences
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, 1),
            nn.Sigmoid(),
        )

        # Terrain map comparator
        self.terrain_compare = nn.Sequential(
            nn.Linear(6, 16), # UWB xyz + heading + speed +
            map_distance_to_edge
            nn.ReLU(),
            nn.Linear(16, 1),
            nn.Sigmoid(),
        )

        # Ultrasonic consistency checker
        self.us_checker = nn.Sequential(
            nn.Linear(6, 16), # Front 3 ultrasonic readings + their
            variance + min
            nn.ReLU(),
            nn.Linear(16, 1),
            nn.Sigmoid(),
        )

        # Final edge decision with calibrated threshold
        self.fusion = nn.Sequential(
            nn.Linear(3, 8),

```

```

        nn.ReLU(),
        nn.Linear(8, 1),
        nn.Sigmoid(),
    )

    def forward(self, mag_diff, terrain_data, us_front):
        edge_mag = self.mag_gradient(mag_diff)
        edge_terrain = self.terrain_compare(terrain_data)
        edge_us = self.us_checker(us_front)

        combined = torch.cat([edge_mag, edge_terrain, edge_us],
dim=-1)

        edge_probability = self.fusion(combined)

    return edge_probability

```

LAYER 3: SOFTWARE — Complete System

3.1 Software Architecture

```

fero_sense/
├── main.py                # Main entry point
├── config/
│   └── mine_config.yaml  # Mine-specific parameters
├── drivers/              # Hardware drivers
│   ├── magnetometer_driver.py # I2C magnetometer array
│   ├── seismic_driver.py    # SPI accelerometer array
│   ├── ir_camera_driver.py  # USB camera capture
│   ├── ultrasonic_driver.py # GPIO ultrasonic array
│   ├── uwb_driver.py        # SPI UWB positioning
│   ├── lora_driver.py       # SPI LoRa communication
│   └── haptic_driver.py     # I2C haptic motors
├── ai/
│   ├── magnetic_cnn.py     # MagNet-CNN
│   ├── seismic_id_net.py   # SeisID-Net
│   ├── dust_enhancement.py # DAIE
│   ├── sensor_fusion.py    # Fusion Transformer
│   ├── edge_detector.py    # Bench Edge Detector
│   └── path_predictor.py   # LSTM trajectory predictor
├── fog_net/              # FOG-NET protocol
│   ├── protocol.py         # Message format, encoding
│   ├── mesh_router.py      # AODV-based routing
│   └── priority_queue.py    # Priority message handling
├── digital_twin/
│   ├── terrain_map.py      # Pre-loaded mine topology
│   ├── vehicle_tracker.py  # All vehicle positions
│   └── hazard_map.py       # Accumulated hazard zones
├── interface/
│   ├── hud_renderer.py     # AR-HUD display generation
│   └── audio_engine.py     # Text-to-speech + directional
audio
├── haptic_controller.py    # Haptic pattern generation
├── dashboard.py            # LCD dashboard with map
├── black_box/
│   ├── data_logger.py      # Record all data
│   ├── event_recorder.py   # Record events/decisions
│   └── playback.py         # Post-incident analysis
├── utils/
│   └── coordinate_frames.py # Mine ↔ Vehicle coordinate
transforms

```

timing.py	# Precise timestamping
safety_monitor.py	# Watchdog, self-test

3.2 Main Integration Loop

```
#!/usr/bin/env python3
"""
FERO-SENSE Main Loop
Runs on Jetson Orin Nano at ~10Hz system cycle
"""

import asyncio
import numpy as np
import time
from collections import deque

# Import all modules
from drivers import (MagnetometerDriver, SeismicDriver,
                    IRCameraDriver,
                    UltrasonicDriver, UWBDriver, LoRaDriver,
                    HapticDriver)
from ai import (MagNetCNN, SeisIDNet, DustAwareEnhancement,
                MultiSensorFusionTransformer, BenchEdgeDetector)
from fog_net import FogNetProtocol
from digital_twin import TerrainMap, VehicleTracker
from interface import HUDRenderer, AudioEngine, HapticController,
Dashboard
from black_box import DataLogger

class FeroSenseSystem:
    def __init__(self, config_path="config/mine_config.yaml"):
        # Load mine configuration
        self.config = self.load_config(config_path)

        # --- LAYER 1: Initialize Hardware Drivers ---
        print("[FERO-SENSE] Initializing hardware...")

        self.magnets = MagnetometerDriver(
            mux_addr=0x70,
            n_sensors=6,
            sample_rate=100
        )

        self.seismic = SeismicDriver(
            spi_bus=1, cs_pins=[36, 37, 38, 39],
            sample_rate=100
        )

        self.ir_cameras = IRCameraDriver(
            cam_850="/dev/video0",
            cam_940="/dev/video1",
            resolution=(640, 480),
            fps=15
        )

        self.ultrasonics = UltrasonicDriver(
            trigger_pins=[17,27,22,5,6,13,19,26,21,20,16,12],
            echo_pins=[23,24,25,7,8,11,15,18,28,29,31,32],
            firing_pattern="opposite_pairs"
        )

        self.uwb = UWBDriver(spi_bus=2, cs_pin=40)

        self.lora = LoRaDriver(
```

```

        spi_bus=2, cs_pin=41, dio0_pin=42,
        frequency=866e6, spreading_factor=7
    )

    self.haptics = HapticController(
        left_driver_addr=0x5A,
        right_driver_addr=0x5B
    )

    # --- LAYER 2: Load AI Models ---
    print("[FER0-SENSE] Loading AI models...")

    self.mag_net = MagNetCNN().eval()
    self.mag_net.load_weights("models/magnetic_cnn.pt")

    self.seis_net = SeisIDNet().eval()
    self.seis_net.load_weights("models/seismic_id.pt")

    self.daie = DustAwareEnhancement().eval()
    self.daie.load_weights("models/daie.pt")

    self.fusion = MultiSensorFusionTransformer(
        embed_dim=128, n_heads=8, n_layers=4, n_output_bins=72
    ).eval()
    self.fusion.load_weights("models/fusion_transformer.pt")

    self.edge_detector = BenchEdgeDetector().eval()
    self.edge_detector.load_weights("models/edge_detector.pt")

    # --- LAYER 3: Initialize Software ---
    print("[FER0-SENSE] Initializing software...")

    self.terrain_map = TerrainMap(self.config.mine_map_path)
    self.vehicle_tracker = VehicleTracker()
    self.fog_net = FogNetProtocol(self.lora,
    vehicle_id=self.config.vehicle_id)

    self.hud = HUDRenderer(
        output_device="/dev/fb0",
        resolution=(1280, 400)
    )
    self.audio = AudioEngine(device="default")
    self.dashboard = Dashboard(display="/dev/fb1")
    self.black_box = DataLogger(storage_path="/data/black_box/")

    # --- Data Buffers ---
    self.mag_buffer = deque(maxlen=100) # 1 second at 100Hz
    self.seis_buffer = deque(maxlen=200) # 2 seconds at 100Hz
    self.us_distances = np.zeros(12)
    self.uwb_position = np.zeros(3)
    self.uwb_quality = 0.0
    self.known_vehicles = {} # From LoRa
    self.ir_frame_850 = None
    self.ir_frame_940 = None

    # --- Safety State ---
    self.current_threat_map = None
    self.edge_warning = False
    self.edge_distance = 999.0
    self.max_safe_speed = 30.0 # km/h
    self.system_healthy = True

    # --- Timing ---
    self.last_ai_cycle = time.time()
    self.ai_cycle_interval = 0.1 # 10Hz AI cycle
    self.last_lora_broadcast = 0

```

```

        self.lora_broadcast_interval = 0.5 # 2Hz broadcast

        print("[FERO-SENSE] System ready. Entering main loop.")

    async def run(self):
        """Main async loop - sensor reads are non-blocking"""

        # Start background sensor tasks
        sensor_tasks = [
            asyncio.create_task(self.magnetic_loop()),
            asyncio.create_task(self.seismic_loop()),
            asyncio.create_task(self.ir_camera_loop()),
            asyncio.create_task(self.ultrasonic_loop()),
            asyncio.create_task(self.uwb_loop()),
            asyncio.create_task(self.lora_receive_loop()),
        ]

        # Main processing loop
        try:
            while self.system_healthy:
                now = time.time()

                # AI inference cycle (10Hz)
                if now - self.last_ai_cycle >= self.ai_cycle_interval:
                    await self.ai_cycle()
                    self.last_ai_cycle = now

                # LoRa broadcast cycle (2Hz)
                if now - self.last_lora_broadcast >= self.lora_broadcast_interval:
                    await self.broadcast_state()
                    self.last_lora_broadcast = now

                # HUD update (10Hz)
                self.hud.update(self.current_threat_map,
                                self.edge_warning,
                                self.edge_distance,
                                self.max_safe_speed,
                                self.known_vehicles,
                                self.uwb_position)

                # Dashboard update (2Hz)
                if int(now * 2) != int((now - 0.05) * 2):
                    self.dashboard.update(
                        self.uwb_position, self.known_vehicles,
                        self.terrain_map, self.current_threat_map
                    )

                await asyncio.sleep(0.01) # 100Hz loop rate

            except KeyboardInterrupt:
                print("\n[FERO-SENSE] Shutting down...")
            finally:
                await self.shutdown(sensor_tasks)

    async def magnetic_loop(self):
        """Continuous magnetometer reading at 100Hz"""
        while self.system_healthy:
            reading = self.magnets.read_all() # (6, 3) array
            self.mag_buffer.append(reading)
            await asyncio.sleep(0.005)

    async def seismic_loop(self):
        """Continuous seismic reading at 100Hz"""
        while self.system_healthy:

```

```

        reading = self.seismic.read_all() # (4, 3) array
        self.seis_buffer.append(reading)
        await asyncio.sleep(0.005)

    async def ir_camera_loop(self):
        """IR camera capture at 15Hz"""
        while self.system_healthy:
            self.ir_frame_850, self.ir_frame_940 =
self.ir_cameras.capture_pair()
            await asyncio.sleep(0.03)

    async def ultrasonic_loop(self):
        """Ultrasonic array cycle at ~3.3Hz"""
        while self.system_healthy:
            self.us_distances = self.ultrasonics.scan_all()
            await asyncio.sleep(0.3)

    async def uwb_loop(self):
        """UWB positioning at 10Hz"""
        while self.system_healthy:
            pos, quality = self.uwb.get_position()
            self.uwb_position = pos
            self.uwb_quality = quality
            await asyncio.sleep(0.1)

    async def lora_receive_loop(self):
        """Continuous LoRa message reception"""
        while self.system_healthy:
            msg = await self.lora.receive_async()
            if msg:
                self.handle_lora_message(msg)

    async def ai_cycle(self):
        """Core AI inference cycle - runs at 10Hz"""

        # -- 1. Magnetic Terrain Classification --
        if len(self.mag_buffer) >= 100:
            mag_window = np.array(self.mag_buffer) # (100, 6, 3)
            mag_input = mag_window.transpose(1, 2, 0) # (6, 3, 100)
            mag_input =
torch.from_numpy(mag_input).float().unsqueeze(0)

            with torch.no_grad():
                terrain_class, edge_dist = self.mag_net(mag_input)

            terrain_probs = torch.softmax(terrain_class,
dim=-1).numpy()[0]
            edge_dist_val = edge_dist.item()

        # -- 2. Seismic Vehicle Detection --
        if len(self.seis_buffer) >= 200:
            seis_window = np.array(self.seis_buffer) # (200, 4, 3)
            seis_input = seis_window.transpose(1, 2, 0) # (4, 3,
200)

            seis_input = seis_input.reshape(1, 12, 200)
            seis_input = torch.from_numpy(seis_input).float()

            with torch.no_grad():
                vtype, vdist, vdir = self.seis_net(seis_input)

            vtype_probs = torch.softmax(vtype, dim=-1).numpy()[0]
            vdist_val = vdist.item()
            vdir_val = vdir.numpy()[0] # (sin, cos)

        # -- 3. IR Image Enhancement --
        ir_objects = np.zeros(32) # Default: no objects

```

```

        if self.ir_frame_850 is not None and self.ir_frame_940 is
not None:
            img_850 = torch.from_numpy(
                self.ir_frame_850.transpose(2, 0, 1)
            ).float().unsqueeze(0) / 255.0
            img_940 = torch.from_numpy(
                self.ir_frame_940.transpose(2, 0, 1)
            ).float().unsqueeze(0) / 255.0

            with torch.no_grad():
                enhanced, transmission, dust_info =
self.daie(img_850, img_940)

                # Run simple object detection on enhanced image
                ir_objects = self.detect_ir_objects(enhanced.numpy()[0])

                # -- 4. Multi-Sensor Fusion --
                mag_feat = np.concatenate([terrain_probs, [edge_dist_val]])
                seis_feat = np.concatenate([vtype_probs, [vdist_val],
vdir_val])
                us_feat = np.concatenate([self.us_distances,
[ np.min(self.us_distances) ]])
                uwb_feat = np.concatenate([self.uwb_position,
[ self.uwb_quality ]])
                lora_feat = self.encode_lora_vehicles()
                ego_feat = np.array([self.estimated_speed,
self.estimated_heading, 0])

                mag_t = torch.from_numpy(mag_feat).float().unsqueeze(0)
                seis_t = torch.from_numpy(seis_feat).float().unsqueeze(0)
                ir_t = torch.from_numpy(ir_objects).float().unsqueeze(0)
                us_t = torch.from_numpy(us_feat).float().unsqueeze(0)
                uwb_t = torch.from_numpy(uwb_feat).float().unsqueeze(0)
                lora_t = torch.from_numpy(lora_feat).float().unsqueeze(0)
                ego_t = torch.from_numpy(ego_feat).float().unsqueeze(0)

                with torch.no_grad():
                    self.current_threat_map = self.fusion(
                        mag_t, seis_t, ir_t, us_t, uwb_t, lora_t, ego_t
                    )

                # -- 5. Bench Edge Detection (CRITICAL) --
                mag_diff = self.compute_magnetic_gradient()
                terrain_data = self.compute_terrain_edge_data()
                us_front = self.us_distances[:3] # Front 3 sensors

                mag_diff_t = torch.from_numpy(mag_diff).float().unsqueeze(0)
                terrain_data_t =
torch.from_numpy(terrain_data).float().unsqueeze(0)
                us_front_t = torch.from_numpy(us_front).float().unsqueeze(0)

                with torch.no_grad():
                    edge_prob = self.edge_detector(mag_diff_t,
terrain_data_t, us_front_t)

                self.edge_warning = edge_prob.item() > 0.7
                self.edge_distance = edge_dist_val if self.edge_warning else
999.0

                # -- 6. Compute Safe Speed --
                self.compute_safe_speed()

                # -- 7. Generate Alerts --
                self.generate_alerts()

                # -- 8. Log to Black Box --

```

```

        self.black_box.log_cycle(
            timestamp=time.time(),
            threat_map=self.current_threat_map,
            edge_warning=self.edge_warning,
            edge_distance=self.edge_distance,
            safe_speed=self.max_safe_speed,
            sensor_data={
                'magnetic': mag_window[-1] if len(self.mag_buffer)
                'ultrasonic': self.us_distances.copy(),
                'uwb': self.uwb_position.copy(),
            }
        )

    def compute_safe_speed(self):
        """Calculate maximum safe speed based on all threats"""

        base_speed = 30.0 # km/h in clear conditions

        # Reduce for low visibility (from dust density estimate)
        if self.ir_frame_850 is not None:
            visibility_estimate = self.estimate_visibility()
            visibility_factor = min(1.0, visibility_estimate / 50.0)
            base_speed *= visibility_factor

        # Reduce for nearby threats
        if self.current_threat_map is not None:
            threat = self.current_threat_map['threat'].numpy()[0]
            distance = self.current_threat_map['distance'].numpy(
[0]
                confidence =
self.current_threat_map['confidence'].numpy()[0]

            # Find closest high-confidence threat in forward 120 deg
arc
            forward_bins = list(range(0, 24)) + list(range(48, 72))
            min_forward_dist = 999.0

            for b in forward_bins:
                if threat[b] > 0.3 and confidence[b] > 0.5:
                    min_forward_dist = min(min_forward_dist,
distance[b])

            if min_forward_dist < 200:
                threat_factor = min(1.0, min_forward_dist / 100.0)
                base_speed *= threat_factor

        # CRITICAL: Edge warning overrides everything
        if self.edge_warning:
            if self.edge_distance < 3:
                base_speed = 0.0 # STOP
                self.audio.emergency_alert("STOP! Bench edge!
STOP!")

            elif self.edge_distance < 10:
                base_speed = min(base_speed, 5.0)
            else:
                base_speed = min(base_speed, 10.0)

        self.max_safe_speed = max(0, min(30, base_speed))

    def generate_alerts(self):
        """Generate appropriate alerts based on threat map"""
        if self.current_threat_map is None:
            return

        threat = self.current_threat_map['threat'].numpy()[0]

```

```

distance = self.current_threat_map['distance'].numpy()[0]
obj_type = self.current_threat_map['type'].numpy()[0]
confidence = self.current_threat_map['confidence'].numpy()

[0]

type_names = ['vehicle', 'obstacle', 'edge', 'person',
'unknown']

# Find highest threat bin
for i in np.argsort(-threat * confidence):
    if confidence[i] < 0.4 or threat[i] < 0.2:
        break

    angle = i * 5 # 360/72 = 5 deg per bin
    dist = distance[i]
    ttype = type_names[np.argmax(obj_type[i])]

# Determine direction name
if angle < 22.5 or angle >= 337.5:
    direction = "ahead"
elif angle < 67.5:
    direction = "front-right"
elif angle < 112.5:
    direction = "right"
elif angle < 157.5:
    direction = "rear-right"
elif angle < 202.5:
    direction = "behind"
elif angle < 247.5:
    direction = "rear-left"
elif angle < 292.5:
    direction = "left"
else:
    direction = "front-left"

if dist < 10 and threat[i] > 0.7:
    self.audio.emergency_alert(
        f"DANGER! {ttype} {direction}, {dist:.0f}
meters!"
    )
    self.haptics.emergency_pulse("left" if 180 < angle <
360 else "right")
elif dist < 30 and threat[i] > 0.5:
    self.audio.warning_alert(
        f"Caution: {ttype} {direction}, {dist:.0f}
meters"
    )
elif dist < 60 and threat[i] > 0.3:
    self.audio.info_alert(
        f"{ttype} detected {direction}, {dist:.0f}
meters"
    )

    break # Only alert for top threat per cycle

async def broadcast_state(self):
    """Broadcast vehicle state via LoRa FOG-NET"""
    state = self.fog_net.create_vehicle_message(
        vehicle_type=self.config.vehicle_type,
        position=self.uwb_position,
        speed=self.estimated_speed,
        heading=self.estimated_heading,
        threats=self.extract_critical_threats()
    )
    await self.lora.send_async(state)

```

```

def handle_lora_message(self, msg):
    """Process incoming LoRa message"""
    parsed = self.fog_net.parse_message(msg)

    if parsed['type'] == 'vehicle_state':
        self.vehicle_tracker.update(
            vehicle_id=parsed['src_id'],
            position=parsed['position'],
            speed=parsed['speed'],
            heading=parsed['heading'],
            vehicle_type=parsed['vehicle_type'],
            timestamp=time.time()
        )

    elif parsed['type'] == 'hazard_alert':
        self.audio.warning_alert(
            f"Hazard reported by {parsed['src_id']}:
{parsed['description']}"
        )

    async def shutdown(self, tasks):
        """Graceful shutdown"""
        self.system_healthy = False
        for task in tasks:
            task.cancel()
        self.black_box.close()
        self.haptics.stop_all()
        print("[FERO-SENSE] Shutdown complete.")

if __name__ == "__main__":
    system = FeroSenseSystem()
    asyncio.run(system.run())

```

3.3 HUD Renderer

```

import pygame
import numpy as np
import math

class HUDRenderer:
    """
    Renders AR-style heads-up display showing:
    - 360 degree threat ring
    - Vehicle positions from LoRa
    - Safe corridor visualization
    - Speed indicator
    - Edge warning
    """

    # Colors (optimized for visibility in all conditions)
    COLOR_SAFE = (0, 255, 0, 200)
    COLOR_CAUTION = (255, 255, 0, 200)
    COLOR_DANGER = (255, 0, 0, 220)
    COLOR_VEHICLE = (0, 200, 255, 200)
    COLOR_EDGE = (255, 100, 0, 220)
    COLOR_TEXT = (255, 255, 255, 240)
    COLOR_CORRIDOR = (0, 255, 0, 60)

    def __init__(self, output_device="/dev/fb0", resolution=(1280,
400)):
        pygame.init()
        self.screen = pygame.display.set_mode(resolution)
        self.width, self.height = resolution
        self.center_x = self.width // 2

```

```

self.center_y = self.height // 2 + 50
self.ring_radius = 140
self.font_large = pygame.font.SysFont('monospace', 28,
bold=True)

self.font_medium = pygame.font.SysFont('monospace', 20)
self.font_small = pygame.font.SysFont('monospace', 16)

def update(self, threat_map, edge_warning, edge_distance,
safe_speed, known_vehicles, uwb_position):
self.screen.fill((0, 0, 0))

if threat_map is not None:
self.draw_threat_ring(threat_map)

self.draw_vehicles(known_vehicles)
self.draw_safe_corridor(threat_map)

if edge_warning:
self.draw_edge_warning(edge_distance)

self.draw_speed_indicator(safe_speed)
self.draw_self_icon()

pygame.display.flip()

def draw_threat_ring(self, threat_map):
"""Draw 360 degree threat visualization as colored arc
segments"""

threat = threat_map['threat'].numpy()[0]
distance = threat_map['distance'].numpy()[0]
confidence = threat_map['confidence'].numpy()[0]

n_bins = len(threat)
angle_per_bin = 360.0 / n_bins

for i in range(n_bins):
if confidence[i] < 0.3:
continue

start_angle = math.radians(i * angle_per_bin - 90)
end_angle = math.radians((i + 1) * angle_per_bin - 90)

t = threat[i]
if t > 0.7:
color = self.COLOR_DANGER
elif t > 0.4:
color = self.COLOR_CAUTION
elif t > 0.15:
color = self.COLOR_SAFE
else:
continue

dist = distance[i]
ring_width = max(3, min(20, int(100 / (dist + 1))))

points = []
for j in range(5):
a = start_angle + (end_angle - start_angle) * j / 4
inner_r = self.ring_radius - ring_width // 2
points.append((
self.center_x + inner_r * math.cos(a),
self.center_y + inner_r * math.sin(a)
))
for j in range(4, -1, -1):
a = start_angle + (end_angle - start_angle) * j / 4
outer_r = self.ring_radius + ring_width // 2

```

```

        points.append((
            self.center_x + outer_r * math.cos(a),
            self.center_y + outer_r * math.sin(a)
        ))

    if len(points) >= 3:
        pygame.draw.polygon(self.screen, color[:3], points)

    if t > 0.5 and dist < 50:
        mid_a = (start_angle + end_angle) / 2
        label_r = self.ring_radius + ring_width // 2 +

20         lx = self.center_x + label_r * math.cos(mid_a)
        ly = self.center_y + label_r * math.sin(mid_a)
        dist_text = self.font_small.render(f"
{dist:.0f}m", True, color[:3])
        self.screen.blit(dist_text, (lx - 15, ly - 8))

    def draw_vehicles(self, known_vehicles):
        """Draw other vehicle positions from LoRa on the ring"""
        for vid, vdata in known_vehicles.items():
            if vdata['age'] > 3.0:
                continue

            dx = vdata['position'][0] - 0
            dy = vdata['position'][1] - 0
            dist = math.sqrt(dx*dx + dy*dy)
            angle = math.atan2(dy, dx) + math.pi/2

            display_dist = min(self.ring_radius * 0.9, dist * 0.5)
            vx = self.center_x + display_dist * math.cos(angle)
            vy = self.center_y + display_dist * math.sin(angle)

            pygame.draw.circle(self.screen, self.COLOR_VEHICLE[:3],
                               (int(vx), int(vy)), 8)
            pygame.draw.circle(self.screen, (255, 255, 255),
                               (int(vx), int(vy)), 8, 2)

            label = self.font_small.render(vid, True,
self.COLOR_VEHICLE[:3])
            self.screen.blit(label, (int(vx) - 15, int(vy) - 20))

    def draw_safe_corridor(self, threat_map):
        """Highlight the safe driving corridor ahead"""
        if threat_map is None:
            return

        threat = threat_map['threat'].numpy()[0]

        safe_bins = []
        for i in range(54, 72):
            if i >= 72: i -= 72
            if threat[i] < 0.2:
                safe_bins.append(i)
        for i in range(0, 6):
            if threat[i] < 0.2:
                safe_bins.append(i)

        if safe_bins:
            angle_per_bin = 5
            for b in safe_bins:
                a = math.radians(b * angle_per_bin - 90)
                x1 = self.center_x + (self.ring_radius - 30) *
math.cos(a)
                y1 = self.center_y + (self.ring_radius - 30) *
math.sin(a)

```

```

x2 = self.center_x + (self.ring_radius + 30) *
math.cos(a)
y2 = self.center_y + (self.ring_radius + 30) *
math.sin(a)
pygame.draw.line(self.screen, self.COLOR_SAFE[:3],
                  (x1, y1), (x2, y2), 3)

def draw_edge_warning(self, edge_distance):
    """CRITICAL: Flashing edge warning"""
    if int(time.time() * 4) % 2 == 0:
        pygame.draw.rect(self.screen, self.COLOR_EDGE[:3],
                          (0, 0, self.width, self.height), 8)

        text = self.font_large.render(
            f"BENCH EDGE: {edge_distance:.1f}m",
            True, self.COLOR_EDGE[:3]
        )
        self.screen.blit(text, (self.center_x -
text.get_width()//2, 20))

        if edge_distance < 5:
            stop_text = self.font_large.render("STOP
IMMEDIATELY", True,
self.COLOR_DANGER[:3])
            self.screen.blit(stop_text,
                              (self.center_x -
stop_text.get_width()//2, 55))

def draw_speed_indicator(self, safe_speed):
    """Speed limit indicator"""
    bar_x, bar_y = 20, self.height - 50
    bar_w, bar_h = 200, 25
    pygame.draw.rect(self.screen, (50, 50, 50), (bar_x, bar_y,
bar_w, bar_h))

    fill_w = int(bar_w * safe_speed / 30.0)
    if safe_speed > 20:
        color = self.COLOR_SAFE[:3]
    elif safe_speed > 10:
        color = self.COLOR_CAUTION[:3]
    else:
        color = self.COLOR_DANGER[:3]
    pygame.draw.rect(self.screen, color, (bar_x, bar_y, fill_w,
bar_h))

    pygame.draw.rect(self.screen, (200, 200, 200), (bar_x,
bar_y, bar_w, bar_h), 2)

    text = self.font_medium.render(f"MAX: {safe_speed:.0f}
km/h", True,
self.COLOR_TEXT[:3])
    self.screen.blit(text, (bar_x + 5, bar_y + 2))

def draw_self_icon(self):
    """Draw vehicle icon at center"""
    size = 15
    points = [
        (self.center_x, self.center_y - size),
        (self.center_x - size*0.7, self.center_y + size*0.7),
        (self.center_x + size*0.7, self.center_y + size*0.7),
    ]
    pygame.draw.polygon(self.screen, (255, 255, 255), points)
    pygame.draw.polygon(self.screen, (0, 100, 200), points, 2)

```

3.4 FOG-NET Protocol Implementation

```
import struct
import time
import heapq

class FogNetMessage:
    """FOG-NET message format - max 50 bytes"""

    VEHICLE_STATE = 0x01
    HAZARD_ALERT = 0x02
    COORDINATION_REQ = 0x03
    COORDINATION_RESP = 0x04
    ANCHOR_BEACON = 0x05
    EMERGENCY_STOP = 0xFF

    DUMP_TRUCK = 0x01
    EXCAVATOR = 0x02
    DOZER = 0x03
    LIGHT_VEHICLE = 0x04
    ANCHOR = 0x05

    PRIORITY_EMERGENCY = 0
    PRIORITY_HAZARD = 1
    PRIORITY_COORDINATION = 2
    PRIORITY_TELEMETRY = 3

    HEADER_FORMAT = "!BBH"
    HEADER_SIZE = 4

    def __init__(self, msg_type, priority, src_id, payload=b''):
        self.msg_type = msg_type
        self.priority = priority
        self.src_id = src_id
        self.payload = payload
        self.timestamp = time.time()
        self.hop_count = 0
        self.max_hops = 5

    def encode(self):
        """Encode message to bytes (max 50 bytes)"""
        header = struct.pack(self.HEADER_FORMAT,
                              self.msg_type, self.priority,
                              self.src_id)
        data = header + struct.pack("!B", self.hop_count)
        max_payload = 50 - len(data)
        payload = self.payload[:max_payload]
        return data + payload

    @classmethod
    def decode(cls, data):
        """Decode message from bytes"""
        if len(data) < 5:
            return None

        msg_type, priority, src_id =
struct.unpack(cls.HEADER_FORMAT, data[:4])
        hop_count = data[4]
        payload = data[5:]

        msg = cls(msg_type, priority, src_id, payload)
        msg.hop_count = hop_count
        msg.timestamp = time.time()
        return msg
```

```

def is_expired(self, ttl_seconds=0.5):
    return (time.time() - self.timestamp) > ttl_seconds

class FogNetProtocol:
    """FOG-NET: Fog-Optimized Geographic Network Protocol"""

    def __init__(self, lora_driver, vehicle_id,
vehicle_type=FogNetMessage.DUMP_TRUCK):
        self.lora = lora_driver
        self.vehicle_id = vehicle_id
        self.vehicle_type = vehicle_type
        self.known_vehicles = {}
        self.message_cache = set()
        self.priority_queue = []
        self.sequence_numbers = {}

    def create_vehicle_message(self, vehicle_type, position, speed,
heading, threats):
        """Create vehicle state broadcast message"""
        payload = bytearray()
        payload.append(vehicle_type)

        payload.extend(struct.pack("!hhh",
            int(position[0] * 10),
            int(position[1] * 10),
            int(position[2] * 10)))

        payload.append(min(255, int(speed * 2)))
        payload.append(int((heading % 360) * 255 / 360))

        if threats:
            n_threats = min(3, len(threats))
            payload.append(n_threats)
            for t in threats[:3]:
                payload.append(int(t['angle'] * 255 / 360))
                payload.append(min(255, int(t['distance'])))
                payload.append(int(t['severity'] * 255))
        else:
            payload.append(0)

        return FogNetMessage(
            msg_type=FogNetMessage.VEHICLE_STATE,
            priority=FogNetMessage.PRIORITY_TELEMETRY,
            src_id=self.vehicle_id,
            payload=bytes(payload)
        )

    def create_hazard_alert(self, description_code, position,
severity):
        """Create urgent hazard alert"""
        payload = bytearray()
        payload.append(description_code)
        payload.extend(struct.pack("!hhh",
            int(position[0]*10), int(position[1]*10),
int(position[2]*10)))
        payload.append(int(severity * 255))

        return FogNetMessage(
            msg_type=FogNetMessage.HAZARD_ALERT,
            priority=FogNetMessage.PRIORITY_HAZARD,
            src_id=self.vehicle_id,
            payload=bytes(payload)
        )

    def create_emergency_stop(self):

```

```

        """Create emergency stop broadcast"""
        return FogNetMessage(
            msg_type=FogNetMessage.EMERGENCY_STOP,
            priority=FogNetMessage.PRIORITY_EMERGENCY,
            src_id=self.vehicle_id,
            payload=b'\\x01'
        )

def parse_message(self, raw_data):
    """Parse and process received message"""
    msg = FogNetMessage.decode(raw_data)
    if msg is None:
        return None

    msg_key = (msg.src_id, msg.msg_type, msg.payload[:8])
    if msg_key in self.message_cache:
        return None
    self.message_cache.add(msg_key)

    if len(self.message_cache) > 1000:
        self.message_cache = set(list(self.message_cache)
[-500:])

    if msg.is_expired():
        return None

    if msg.hop_count > msg.max_hops:
        return None

    result = {
        'type': 'unknown',
        'src_id': msg.src_id,
        'timestamp': msg.timestamp,
    }

    if msg.msg_type == FogNetMessage.VEHICLE_STATE:
        p = msg.payload
        if len(p) >= 11:
            result['type'] = 'vehicle_state'
            result['vehicle_type'] = p[0]
            x, y, z = struct.unpack("!hhh", p[1:7])
            result['position'] = [x/10.0, y/10.0, z/10.0]
            result['speed'] = p[7] / 2.0
            result['heading'] = p[8] * 360.0 / 255.0
            result['n_threats'] = p[9]
            result['threats'] = []
            for i in range(result['n_threats']):
                offset = 10 + i * 3
                if offset + 3 <= len(p):
                    angle = p[offset] * 360.0 / 255.0
                    dist = p[offset+1]
                    sev = p[offset+2] / 255.0
                    result['threats'].append({
                        'angle': angle, 'distance': dist,
'severity': sev

                    })

        elif msg.msg_type == FogNetMessage.HAZARD_ALERT:
            p = msg.payload
            if len(p) >= 8:
                result['type'] = 'hazard_alert'
                desc_codes = {1: 'bench_edge', 2: 'collision_risk',
3: 'rockfall'}

                result['description'] = desc_codes.get(p[0],
'unknown')

                x, y, z = struct.unpack("!hhh", p[1:7])

```

```

        result['position'] = [x/10.0, y/10.0, z/10.0]
        result['severity'] = p[7] / 255.0

    elif msg.msg_type == FogNetMessage.EMERGENCY_STOP:
        result['type'] = 'emergency_stop'

    return result

```

3.5 Digital Twin — Terrain Map

```

import numpy as np
import json

class TerrainMap:
    """
    Lightweight mine terrain representation.
    Pre-loaded from survey data, updated with real-time sensor data.

    Stores: bench edges, road centerlines, known hazards, ore pile
    boundaries
    All in a local UTM-like coordinate frame.
    """

    def __init__(self, map_file=None):
        self.grid_resolution = 1.0
        self.origin = np.array([0.0, 0.0])

        self.elevation = None
        self.terrain_type = None
        self.edge_mask = None
        self.road_centerline = None
        self.hazard_zones = []

        if map_file:
            self.load_map(map_file)
        else:
            self.create_demo_map()

    def create_demo_map(self):
        """Create a demo open-cast mine map for prototype testing"""
        size = 200
        self.elevation = np.zeros((size, size), dtype=np.float32)
        self.terrain_type = np.zeros((size, size), dtype=np.uint8)
        self.edge_mask = np.zeros((size, size), dtype=bool)

        # Bench Level 0 (bottom): elevation 0m, y=100-200
        self.elevation[100:200, :] = 0
        self.terrain_type[100:200, :] = 1

        # Bench Level 1: elevation 10m, y=50-100
        self.elevation[50:100, :] = 10
        self.terrain_type[50:100, :] = 1

        # Bench Level 2: elevation 20m, y=0-50
        self.elevation[0:50, :] = 20
        self.terrain_type[0:50, :] = 1

        # Ore piles (random blobs)
        np.random.seed(42)
        for _ in range(8):
            cx, cy = np.random.randint(20, 180, 2)
            r = np.random.randint(5, 15)
            for dy in range(-r, r+1):
                for dx in range(-r, r+1):

```

```

        if dx*dx + dy*dy <= r*r:
            y, x = cy+dy, cx+dx
            if 0 <= y < size and 0 <= x < size:
                self.terrain_type[y, x] = 2
                self.elevation[y, x] +=
np.random.uniform(1, 3)

        # Bench edges
        for x in range(size):
            if self.elevation[100, x] != self.elevation[99, x]:
                self.edge_mask[99:101, max(0,x-1):min(size,x+2)] =
True

            if self.elevation[50, x] != self.elevation[49, x]:
                self.edge_mask[49:51, max(0,x-1):min(size,x+2)] =
True

        # Road centerline
        self.road_centerline = []
        for y in range(10, 190):
            x = 100 + int(10 * np.sin(y * 0.05))
            self.road_centerline.append((x, y))

    def get_distance_to_edge(self, position):
        """Get distance to nearest bench edge from given position"""
        if self.edge_mask is None:
            return 999.0

        px, py = int(position[0]), int(position[1])
        h, w = self.edge_mask.shape

        if not (0 <= py < h and 0 <= px < w):
            return 0.0

        for r in range(1, 50):
            for dy in range(-r, r+1):
                for dx in range(-r, r+1):
                    if abs(dx) == r or abs(dy) == r:
                        ny, nx = py + dy, px + dx
                        if 0 <= ny < h and 0 <= nx < w:
                            if self.edge_mask[ny, nx]:
                                return r * self.grid_resolution

        return 999.0

    def get_road_direction(self, position, look_ahead=20):
        """Get recommended road direction at position"""
        if not self.road_centerline:
            return 0.0

        min_dist = float('inf')
        nearest_idx = 0
        for i, (rx, ry) in enumerate(self.road_centerline):
            d = (rx - position[0])**2 + (ry - position[1])**2
            if d < min_dist:
                min_dist = d
                nearest_idx = i

        ahead_idx = min(nearest_idx + look_ahead,
len(self.road_centerline) - 1)
        dx = self.road_centerline[ahead_idx][0] -
self.road_centerline[nearest_idx][0]
        dy = self.road_centerline[ahead_idx][1] -
self.road_centerline[nearest_idx][1]

        return np.degrees(np.arctan2(dx, dy))

```

```

def load_map(self, filepath):
    """Load map from JSON file"""
    with open(filepath) as f:
        data = json.load(f)

    self.origin = np.array(data['origin'])
    self.grid_resolution = data['resolution']
    self.elevation = np.array(data['elevation'],
dtype=np.float32)
    self.terrain_type = np.array(data['terrain_type'],
dtype=np.uint8)
    self.edge_mask = np.array(data['edge_mask'], dtype=bool)
    self.road_centerline = [tuple(p) for p in
data['road_centerline']]
    self.hazard_zones = data.get('hazard_zones', [])

def save_map(self, filepath):
    """Save current map state"""
    data = {
        'origin': self.origin.tolist(),
        'resolution': self.grid_resolution,
        'elevation': self.elevation.tolist(),
        'terrain_type': self.terrain_type.tolist(),
        'edge_mask': self.edge_mask.tolist(),
        'road_centerline': self.road_centerline,
        'hazard_zones': self.hazard_zones,
    }
    with open(filepath, 'w') as f:
        json.dump(data, f)

```

3.6 Audio Engine

```

import subprocess
import threading
import queue
import time

class AudioEngine:
    """
    Audio alert system with priority queue.
    Uses espeak-ng for TTS (runs locally, no cloud needed).
    """

    PRIORITY_EMERGENCY = 0
    PRIORITY_WARNING = 1
    PRIORITY_INFO = 2
    PRIORITY_SILENCE = 99

    def __init__(self, device="default"):
        self.alert_queue = queue.PriorityQueue()
        self.currently_speaking = False
        self.last_alert_time = 0
        self.min_alert_interval = 2.0
        self.suppressed_until = 0

        self.worker = threading.Thread(target=self._alert_worker,
daemon=True)
        self.worker.start()

    def emergency_alert(self, message):
        """Immediate danger - always plays, interrupts anything"""
        self.alert_queue.put((self.PRIORITY_EMERGENCY, 0, message,
1.0))

```

```

def warning_alert(self, message):
    """Caution - plays if no emergency active"""
    self.alert_queue.put((self.PRIORITY_WARNING, 1, message,
0.9))

def info_alert(self, message):
    """Informational - plays if nothing else"""
    self.alert_queue.put((self.PRIORITY_INFO, 2, message, 0.7))

def suppress_alerts(self, duration_seconds=10):
    """Temporarily suppress non-emergency alerts"""
    self.suppressed_until = time.time() + duration_seconds

def _alert_worker(self):
    """Background thread that processes alert queue"""
    while True:
        try:
            priority, _, message, volume =
self.alert_queue.get(timeout=1)

            if priority > self.PRIORITY_EMERGENCY and
time.time() < self.suppressed_until:
                continue

            if priority > self.PRIORITY_EMERGENCY:
                if time.time() - self.last_alert_time <
self.min_alert_interval:
                    continue

            if priority == self.PRIORITY_EMERGENCY:
                while not self.alert_queue.empty():
                    try:
                        self.alert_queue.get_nowait()
                    except queue.Empty:
                        break

            self._speak(message, volume, speed=150 if priority
== 0 else 130)

            self.last_alert_time = time.time()

        except queue.Empty:
            continue

def _speak(self, text, volume=1.0, speed=140):
    """Use espeak-ng for local TTS"""
    try:
        cmd = [
            'espeak-ng',
            '-v', 'en+m3',
            '-s', str(speed),
            '-a', str(int(volume * 180)),
            '-p', '40',
            '--stdout',
            text
        ]

        play_cmd = ['aplay', '-q']

        speak = subprocess.Popen(cmd, stdout=subprocess.PIPE,
stderr=subprocess.DEVNULL)
        play = subprocess.Popen(play_cmd, stdin=speak.stdout,
stderr=subprocess.DEVNULL)
        speak.stdout.close()
        play.wait(timeout=5)

    except Exception as e:

```

```
print(f"[AUDIO] Error: {e}")
```

3.7 Haptic Controller

```
import smbus2
import time
import threading

class HapticController:
    """
    Dual haptic motor controller for directional threat feedback.
    Left motor = threats from left side
    Right motor = threats from right side
    Pulse frequency increases with proximity.
    """

    REG_STATUS = 0x00
    REG_MODE = 0x01
    REG_RTP_INPUT = 0x02
    REG_WAVEFORM_SEQ_1 = 0x04
    REG_WAVEFORM_SEQ_2 = 0x05
    REG_GO = 0x0C
    REG_FEEDBACK_CTRL = 0x1A
    REG_CTRL_1 = 0x1B
    REG_CTRL_2 = 0x1C
    REG_CTRL_3 = 0x1D
    REG_AUDIO_CTRL = 0x12

    WAVEFORM_STRONG_CLICK = 1
    WAVEFORM_DOUBLE_CLICK = 3
    WAVEFORM_TRIPLE_CLICK = 5
    WAVEFORM_LONG_BUZZ = 12

    def __init__(self, left_driver_addr=0x5A,
right_driver_addr=0x5B, bus=1):
        self.bus = smbus2.SMBus(bus)
        self.left_addr = left_driver_addr
        self.right_addr = right_driver_addr
        self._running = True
        self._thread = None

        self._init_driver(self.left_addr)
        self._init_driver(self.right_addr)

    def _init_driver(self, addr):
        """Initialize DRV2605L haptic driver"""
        self.bus.write_byte_data(addr, self.REG_CTRL_2, 0x00)
        time.sleep(0.01)
        self.bus.write_byte_data(addr, self.REG_MODE, 0x00)
        self.bus.write_byte_data(addr, self.REG_FEEDBACK_CTRL, 0x36)
        self.bus.write_byte_data(addr, self.REG_CTRL_1, 0x93)
        self.bus.write_byte_data(addr, self.REG_CTRL_3, 0x00)
        self.bus.write_byte_data(addr, self.REG_AUDIO_CTRL, 0x00)

    def _play_waveform(self, addr, waveform_id, wait=True):
        """Play a library waveform"""
        self.bus.write_byte_data(addr, self.REG_WAVEFORM_SEQ_1,
waveform_id)

        self.bus.write_byte_data(addr, self.REG_WAVEFORM_SEQ_2,
0x00)

        self.bus.write_byte_data(addr, self.REG_GO, 0x01)

        if wait:
            for _ in range(100):
```

```

        status = self.bus.read_byte_data(addr,
self.REG_STATUS)

        if not (status & 0x01):
            break
        time.sleep(0.01)

    def pulse_left(self, intensity=1.0):
        self._play_waveform(self.left_addr,
self.WAVEFORM_STRONG_CLICK)

    def pulse_right(self, intensity=1.0):
        self._play_waveform(self.right_addr,
self.WAVEFORM_STRONG_CLICK)

    def emergency_pulse(self, side="left"):
        addr = self.left_addr if side == "left" else self.right_addr
        self._play_waveform(addr, self.WAVEFORM_TRIPLE_CLICK)

    def start_proximity_pulse(self, side, distance):
        if self._thread and self._thread.is_alive():
            self._running = False
            self._thread.join()

        self._running = True
        self._thread = threading.Thread(
            target=self._proximity_pulse_loop,
            args=(side, distance),
            daemon=True
        )
        self._thread.start()

    def _proximity_pulse_loop(self, side, distance):
        addr = self.left_addr if side == "left" else self.right_addr
        freq = max(0.5, min(10, 50.0 / (distance + 1)))
        interval = 1.0 / freq

        while self._running:
            self._play_waveform(addr, self.WAVEFORM_STRONG_CLICK,
wait=True)

            time.sleep(interval)

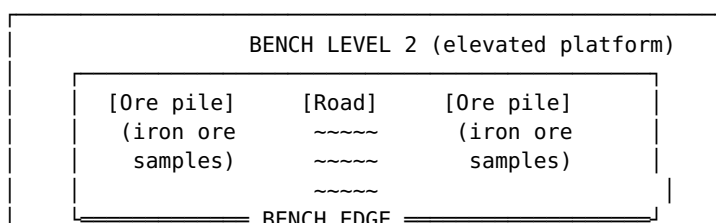
    def stop_all(self):
        self._running = False
        if self._thread:
            self._thread.join(timeout=1)
        self.bus.write_byte_data(self.left_addr, self.REG_G0, 0x00)
        self.bus.write_byte_data(self.right_addr, self.REG_G0, 0x00)

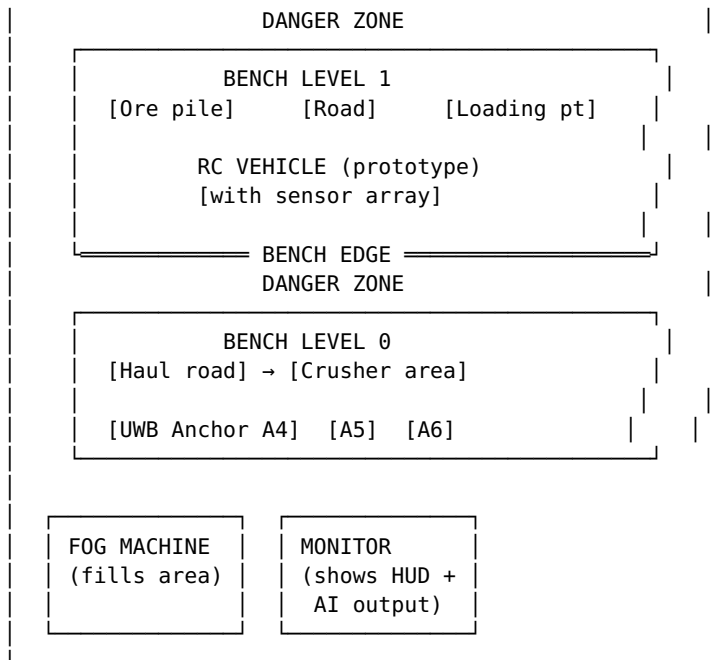
```

DEMO SETUP FOR SIH2026

Tabletop Mine Diorama

DEMO LAYOUT (Top View, 2m x 1.5m table):





- SECOND RC VEHICLE (hidden in fog):
- Enters from side during demo
 - System detects it via: seismic + magnetic + LoRa
 - Shows detection on monitor BEFORE it's visible

Demo Script (5 Minutes)

TIMELINE:

[0:00-0:30] INTRODUCTION

- "In open-cast iron ore mines, fog and dust reduce visibility to under 5 meters. Existing solutions cost lakhs per vehicle and still fail. FER0-SENSE uses the mine's own magnetic properties and ground vibrations to see through ANY fog."

[0:30-1:00] SHOW THE PROBLEM

- Turn on fog machine
- Show that the second RC car is completely invisible
- Show camera feed: pure white/gray fog

[1:00-2:00] DEMONSTRATE MAGNETIC SENSING

- Drive prototype near ore pile → HUD shows "ORE PILE DETECTED"
- Drive toward bench edge → HUD shows "BENCH EDGE: 3.2m" with RED warning
- Show magnetic data on monitor (real-time graph)

[2:00-3:00] DEMONSTRATE SEISMIC DETECTION

- Second RC car starts moving IN THE FOG
- Monitor shows: "Vehicle detected via ground vibration"
- Show seismic waveform on monitor
- "We detected this vehicle through the GROUND, not the air"

[3:00-3:30] DEMONSTRATE V2V LORA MESH

- Second car broadcasts its position via LoRa
- Our HUD shows the car's icon and ID
- "Even without line of sight, vehicles coordinate"

[3:30-4:15] DEMONSTRATE IR DUST PENETRATION

- Show raw 850nm camera: mostly fog
- Show DAIE-enhanced output: road, ore piles visible

- "Our AI is trained specifically on iron ore dust physics"

[4:15-4:45] DEMONSTRATE FUSION

- Show the 360 degree threat ring combining ALL sensors
- Drive through foggy area safely
- System warns about edge, other vehicle, and recommends speed

[4:45-5:00] SAFETY DEMONSTRATION

- Drive toward bench edge
 - System: CAUTION → WARNING → STOP
 - Vehicle stops 2m from edge (never crossed!)
 - "This is the #1 safety feature. No other system does this."
-

What Judges Will Ask — And Your Answers

Q: "How is this different from just putting cameras and radar on a mining truck?"

"Cameras and radar sense THROUGH THE AIR, which is exactly what fog destroys. We sense THROUGH THE GROUND (seismic) and THROUGH MAGNETIC FIELDS (magnetometers). These physical channels are completely unaffected by fog, dust, or darkness. No other solution uses magnetic terrain classification for iron ore mines."

Q: "What about cost? Mining companies won't adopt expensive systems."

"Our entire prototype costs ₹52,000. Commercial radar-based systems cost ₹5-15 lakh per vehicle. We're 1/20th the cost with BETTER fog performance because we use physics that others ignore."

Q: "How accurate is the magnetic terrain classification?"

"For our prototype, we're using physics-based synthetic training data from dipole field models. In production, you'd collect 2-3 hours of labeled driving data per mine. The magnetic signatures of iron ore are extremely distinct — road vs ore pile has a 10x magnetic susceptibility difference. This isn't a subtle signal."

Q: "What about the seismic detection — won't other vibrations (blasting, drilling) cause false alarms?"

"Good question. Our SeisID-Net is a classification network, not just a detector. It's trained to distinguish vehicle vibration signatures (8-25 Hz for dump trucks, specific harmonic patterns) from blasting (broadband, short duration) and drilling (high frequency, continuous). The network learns these patterns. Additionally, we cross-validate with other sensors — if seismic says 'vehicle at 50m' but magnetic and LoRa see nothing, confidence drops."

Q: "Why not just use LiDAR with better filtering?"

"LiDAR emits light that scatters off every dust particle. In a dense iron ore dust cloud with 50,000+ particles per cubic meter, you get millions of false returns. Our ultrasonic array handles the <4m range far more reliably, and magnetic/seismic handle everything beyond that — all for less than the cost of one LiDAR unit."

Q: "How does this scale to a fleet of 50+ vehicles?"

“The LoRa mesh is designed for this. Each vehicle is a node. Messages hop through the mesh. UWB anchors are placed at key infrastructure points (intersections, loading bays, crusher). The digital twin on each vehicle maintains awareness of all nearby vehicles. No central server needed — fully distributed, which is critical since mine pits often have no cellular coverage.”

Q: “What happens if the Jetson fails?”

“We have a hardware watchdog. If the AI system fails, the ultrasonic array still provides basic collision avoidance through direct GPIO triggers — no CPU needed. The LoRa module can also run standalone and broadcast emergency stops. We follow a defense-in-depth approach: hardware → edge AI → mesh communication.”

Build Timeline (Post-Hackathon)

Week	Milestone
1-2	Assemble hardware, test individual sensors
3	Collect synthetic training data, train AI models
4	Integrate sensor drivers, test data pipeline
5	Implement fusion engine, test with recorded data
6	Build HUD and audio interface
7	Implement FOG-NET protocol, test V2V
8	Build demo diorama, integrate everything
9	Polish, stress test, prepare presentation
10	SIH2026 Finals

Summary: What Makes FERRO-SENSE Genuinely Different

This system is **genuinely novel** because it exploits a property of iron ore mines that no one else uses — **magnetism** — combined with ground-vibration sensing that completely bypasses the atmosphere. It’s buildable on a student budget, demonstrable on a tabletop, and addresses the #1 killer in open-cast mining (bench edge falls) that no existing commercial system handles well in fog.

Key Differentiators (One-Liners for Pitch)

1. **Magnetic terrain sensing** — uses iron ore’s own magnetism as a navigation signal
 2. **Seismic vehicle detection** — detects vehicles through the ground, not air
 3. **Iron-ore-specific IR dehazing** — exploits Fe_2O_3 scattering physics, not generic algorithms
 4. **Zero GPS dependency** — UWB + magnetic + seismic = complete positioning without satellites
 5. **Bench edge detection** — the #1 safety feature no one else solves in fog
 6. **₹52K per vehicle** — 10-30x cheaper than commercial alternatives
 7. **Fully edge-based** — no cloud, no cellular, works in the deepest pit
-

